



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Análisis Sintáctico

(Informe de implementación)

Profesor:

Ing. Félix Márquez

Alumnos:

Victor Canchis V-27.935.276

Blas Gonzalez V-31.074.243

Diosdado Muñoz V-30.932.108

Celimar Rojas V-31.981.398

Ciudad Guayana, julio de 2026

Índice

Introduccion	4
Diseño del Lexer y Parser para Docker Compose	6
<i>Descripción del Trabajo Realizado.....</i>	<i>6</i>
<i>Explicación de los Programas Implementados.....</i>	<i>6</i>
<i>Gramática Base</i>	<i>6</i>
<i>Fragmento de Código</i>	<i>7</i>
<i>Analizador en Python (Script Principal).....</i>	<i>10</i>
<i>Analizadores en Lenguajes Adicionales.....</i>	<i>13</i>
<i>Visualizador de Datos</i>	<i>13</i>
<i>Tabla Comparativa de Tiempos de Ejecución</i>	<i>13</i>
<i>Conclusiones del Rendimiento Analítico.....</i>	<i>14</i>
Asistente de Programación Híbrido	16
<i>Descripción del trabajo realizado</i>	<i>16</i>
<i>Explicación de cada programa</i>	<i>18</i>
<i>tokens.py — Estructuras de datos del sistema léxico</i>	<i>18</i>
<i>similarity.py — Métrica de confianza</i>	<i>18</i>
<i>llm_fallback.py — Simulación de la consulta a IA.....</i>	<i>19</i>
<i>lexer.py — Motor del pipeline híbrido.....</i>	<i>19</i>
<i>main.py — Punto de entrada</i>	<i>20</i>

Gráficas de tiempo de ejecución para n pruebas	21
Descripción del trabajo realizado (Fase Sintáctica)	22
<i>Analizador Sintáctico Recursivo Descendente (Parser LL1)</i>	<i>23</i>
<i>Árbol de Sintaxis Abstracta (AST)</i>	<i>23</i>
<i>Flujo de Integración (Lexer-Parser)</i>	<i>24</i>
Explicación de cada programa (Fase Sintáctica)	25
<i>Módulo ast_nodos.py: Estructuras de Datos del Árbol</i>	<i>25</i>
<i>Fragmento representativo:</i>	<i>26</i>
<i>Módulo parser_uneg.py: El Motor Sintáctico</i>	<i>26</i>
<i>Módulo main.py: Integración y Salida Final</i>	<i>27</i>
<i>Análisis de las gráficas:</i>	<i>30</i>
Discusión de Resultados	31

Introducción

El análisis sintáctico constituye una de las fases fundamentales en el proceso de compilación e interpretación de lenguajes formales. Su propósito es verificar que la secuencia de tokens generada por el analizador léxico se ajuste a una gramática predefinida y, a partir de ella, construir una representación jerárquica que refleje la estructura lógica del programa o archivo de configuración. Esta representación, normalmente en forma de árbol de sintaxis abstracta (AST), es la base sobre la que se apoyan las fases posteriores de análisis semántico y generación de código.

En el presente trabajo se abordan dos vertientes complementarias del análisis sintáctico. La primera consiste en el diseño y evaluación de un analizador sintáctico para archivos `docker-compose.yml`, utilizando la herramienta ANTLR4 para generar parsers en tres lenguajes de programación distintos (Java, Python y JavaScript). El objetivo de esta parte es medir el rendimiento de cada implementación y comparar la influencia de la plataforma de ejecución en los tiempos de procesamiento, especialmente en escenarios de ejecución por lotes y con arranque en frío.

La segunda vertiente se centra en el desarrollo de un asistente de programación híbrido para un subconjunto de Python denominado UnegScript. Se implementa un lexer que no solo reconoce patrones léxicos mediante expresiones regulares, sino que también incorpora un mecanismo de corrección automática basado en distancia de Levenshtein y un fallback simulado de inteligencia artificial para resolver errores tipográficos. Sobre este flujo léxico limpio se construye un analizador sintáctico recursivo descendente (LL(1)) que genera un AST, demostrando así la viabilidad de un pipeline completo de análisis.

El informe se organiza en dos grandes bloques experimentales, cada uno con su propia metodología, resultados y gráficas de tiempos de ejecución. Finalmente, se presenta una discusión consolidada que integra las conclusiones de ambos enfoques, destacando las fortalezas y limitaciones de las técnicas empleadas, así como las posibles líneas de mejora.

Diseño del Lexer y Parser para Docker Compose

El analizador sintáctico (parser) es el núcleo del proceso de compilación, encargado de transformar los tokens en estructuras jerárquicas que definen la lógica y validez de un programa o archivo de configuración. En esta sección se detalla la creación de un lexer y parser destinados a procesar un archivo docker compose, enfocándose en describir las interfaces de red del sistema. Adicionalmente, se ejecuta un experimento de carga para evaluar cómo la diferencia en la implementación y optimización de código entre distintos lenguajes de programación impacta directamente en los tiempos de ejecución del análisis.

Descripción del Trabajo Realizado

El desarrollo partió del diseño de una Gramática Libre de Contexto (GLC) unificada, utilizando el metacompilador ANTLR4 para abstraer las reglas propias del formato YAML (tales como los bloques **networks:**, subredes y controladores). Con esta base, se generaron al menos tres parsers implementados en lenguajes diferentes (como Python, Java y C/Rust).

Para el experimento de carga, se estructuró un entorno de prueba con n archivos, cumpliendo rigurosamente con el parámetro establecido de ejecutar el análisis con una cantidad de $5 < n < 20$ archivos. Estos archivos fueron sometidos a evaluación iterativa por cada uno de los analizadores construidos para cronometrar su rendimiento.

Explicación de los Programas Implementados

Gramática Base

Este archivo centraliza las reglas léxicas (manejo de cadenas, identificadores y valores sin comillas) y las reglas sintácticas descendentes necesarias para estructurar el Árbol de Sintaxis Abstracta de las redes de Docker.

Fragmento de Código

```
grammar DockerCompose;

// ----- Parser Rules -----

compose_file
    : version? services? networks? EOF
    ;

version
    : VERSION COLON value_item
    ;

services
    : SERVICES COLON service+
    ;

service
    : IDENTIFIER COLON service_body
    ;

service_body
    : networks_ref*
    ;

networks_ref
    : NETWORKS COLON network_ref_list+
    ;

network_ref_list
    : HYPHEN IDENTIFIER
    ;

networks
    : NETWORKS COLON network+
```

```

;

network
    : IDENTIFIER COLON network_body
    ;

network_body
    : (driver_spec | ipam_spec | external_spec | internal_spec)*
    ;

driver_spec
    : DRIVER COLON value_item
    ;

ipam_spec
    : IPAM COLON ipam_body
    ;

ipam_body
    : (driver_ipam_spec | config_spec)*
    ;

driver_ipam_spec
    : DRIVER COLON value_item
    ;

config_spec
    : CONFIG COLON config_item+
    ;

config_item
    : HYPHEN ipam_config_item+
    ;

ipam_config_item
    : subnet_spec
    | gateway_spec
    | ip_range_spec
    ;

```



```

subnet_spec
    : SUBNET COLON value_item
    ;

gateway_spec
    : GATEWAY COLON value_item
    ;

ip_range_spec
    : IP_RANGE COLON value_item
    ;

external_spec
    : EXTERNAL COLON value_item
    ;

internal_spec
    : INTERNAL COLON value_item
    ;

// Regla comodín para aceptar cualquier tipo de valor
value_item
    : STRING | VALUE | IDENTIFIER | BOOLEAN
    ;

// ----- Lexer Rules -----
VERSION : 'version' ;
SERVICES : 'services' ;
NETWORKS : 'networks' ;
DRIVER : 'driver' ;
IPAM : 'ipam' ;
CONFIG : 'config' ;
SUBNET : 'subnet' ;
GATEWAY : 'gateway' ;
IP_RANGE : 'ip_range' ;
EXTERNAL : 'external' ;

```

```

INTERNAL : 'internal' ;
COLON : ':' ;
HYPHEN : '-' ;

BOOLEAN : 'true' | 'false' ;
IDENTIFIER : [a-zA-Z_][a-zA-Z0-9_\-]* ;
VALUE : [a-zA-Z0-9_\-./]+ ;
STRING : '"' ~["]* '"' | '\'' ~[']* '\'' ;
WS : [ \t\r\n]+ -> skip ;
COMMENT : '#' ~[\r\n]* -> skip ;

```

Analizador en Python (Script Principal)

Este programa invoca el parser generado e implementa un manejador de errores personalizado para la captura controlada de fallos. El script lee los archivos, mide el tiempo exacto de procesamiento e implementa una gestión de registros estricta: los resultados se guardan en la carpeta donde está el script y solo contienen el registro de salud y el tiempo, sin palabras redundantes.

```

import sys
import time
import os

from antlr4 import *
from antlr4.error.ErrorListener import ErrorListener as ANTLRErrorListener
from DockerComposeLexer import DockerComposeLexer
from DockerComposeParser import DockerComposeParser

class ErrorListener(ANTLR4ErrorListener):
    """Listener personalizado para capturar errores sintácticos y
    léxicos."""
    def __init__(self):

```

```

        self.errors = []

    def syntaxError(self, recognizer, offendingSymbol, line, column, msg,
e):

        self.errors.append(f"Línea {line}:{column} - {msg}")

        def reportAmbiguity(self, recognizer, dfa, startIndex, stopIndex,
exact, ambigAlts, configs):

            pass

        def reportAttemptingFullContext(self, recognizer, dfa, startIndex,
stopIndex, conflictingAlts, configs):

            pass

        def reportContextSensitivity(self, recognizer, dfa, startIndex,
stopIndex, prediction, configs):

            pass

def analyze_file(filepath):

    """Analiza el archivo dado y retorna (exitoso, mensaje,
tiempo_segundos)."""

    if not os.path.isfile(filepath):

        return False, f"Archivo no encontrado: {filepath}", 0.0

    try:

        with open(filepath, 'r', encoding='utf-8') as f:

            content = f.read()

    except Exception as e:

        return False, f"Error al leer el archivo: {str(e)}", 0.0

    input_stream = InputStream(content)
    lexer = DockerComposeLexer(input_stream)
    stream = CommonTokenStream(lexer)
    parser = DockerComposeParser(stream)

    error_listener = ErrorListener()
    parser.removeErrorListeners()

```

```

parser.addErrorListener(error_listener)

start_time = time.perf_counter()

try:
    tree = parser.compose_file()
except Exception as e:
    elapsed = time.perf_counter() - start_time
    return False, f"Excepción durante el parseo: {str(e)}", elapsed
elapsed = time.perf_counter() - start_time

if error_listener.errors:
    error_msg = "; ".join(error_listener.errors)
    return False, error_msg, elapsed

return True, "Análisis sintáctico completado", elapsed

def main():
    if len(sys.argv) < 2:
        print("Uso: python analyze_compose.py  
<archivo_docker_compose.yml>")
        sys.exit(1)

    filepath = sys.argv[1]
    success, message, elapsed = analyze_file(filepath)

    log_dir = os.path.dirname(os.path.abspath(__file__))
    log_file = os.path.join(log_dir, "analysis.log")

    try:
        with open(log_file, 'a', encoding='utf-8') as f:

```

```

        f.write(f"{os.path.basename(filepath)} | SALUD: {'EXITOSO' if
success else 'FALLIDO'} | {elapsed:.6f}s\n")
    except Exception as e:
        print(f"Error al escribir el log: {e}", file=sys.stderr)

print(f"Análisis completado. Ver log en: {log_file}")
if success:
    print("✅ Parseo exitoso")
else:
    print(f"❌ Parseo con errores: {message}")

if __name__ == "__main__":
    main()

```

Analizadores en Lenguajes Adicionales

Replican la lógica del script base operando bajo sus propios ecosistemas (como compiladores nativos o máquinas virtuales) procesando los archivos requeridos para medir la diferencia de implementación que impacta los tiempos de ejecución del experimento.

Visualizador de Datos

Script complementario encargado de consolidar los archivos de registro de las tres implementaciones y generar una representación visual para comparar el rendimiento.

Tabla Comparativa de Tiempos de Ejecución

A continuación se presentan los resultados obtenidos tras someter la gramática ANTLR4 a un lote

de pruebas con archivos `docker-compose.yml`. El objetivo de la métrica es evaluar el rendimiento en la clasificación léxica y sintáctica utilizando tres entornos distintos. Los tiempos están expresados en milisegundos (ms).

<i>Archivo de Prueba</i>	<i>Estado de Sintaxis</i>	<i>Tiempo Java (ms)</i>	<i>Tiempo Python (ms)</i>	<i>Tiempo JavaScript (ms)</i>
<i>prueba1.yml</i>	ERROR	315 *	4	11
<i>prueba2.yml</i>	ERROR	78	5	13
<i>prueba3.yml</i>	OK	74	11	11
<i>prueba4.yml</i>	OK	76	6	14
<i>prueba5.yml</i>	OK	75	6	15
<i>prueba6.yml</i>	ERROR	74	4	13
Promedio de Ejecución		~115 ms	~6 ms	~13 ms

** Nota técnica: El tiempo de ejecución del primer archivo en Java incluye la fuerte penalización del arranque en frío de la JVM (Cold Start)*

Conclusiones del Rendimiento Analítico

A partir del log de resultados recolectados por el controlador de PowerShell, se extraen las siguientes conclusiones fundamentales para la viabilidad del proyecto:

- **Consistencia Multiplataforma de ANTLR4:** Se comprobó el éxito absoluto en la portabilidad del código generado por ANTLR. Indistintamente del entorno de ejecución (JVM, CPython o motor V8 de Node.js), el árbol de análisis sintáctico

clasificó de manera idéntica los archivos válidos y erróneos. La gramática diseñada es robusta.

- **La Eficiencia de Python en Entornos "Short-lived":** Para este contexto específico de pruebas por lotes donde cada script nace y muere rápidamente (scripts de vida corta), Python demostró ser el entorno más ágil. Dominó el experimento con un promedio asombroso de ~6 ms por archivo, resaltando su muy baja sobrecarga de inicialización.
- **JavaScript (Node.js) como Ecosistema Equilibrado:** El motor de Node.js ofreció un rendimiento sumamente estable. Con un promedio constante de ~13 ms, procesó los árboles asíncronamente con gran eficacia, sin sufrir penalizaciones de arranque abruptas. Es una alternativa robusta para integraciones web.
- **El Efecto "Cold Start" en la Arquitectura Java:** Aunque Java cuenta con una potencia de cómputo inmensa, las métricas revelaron su debilidad principal en este escenario: el arranque de la Máquina Virtual de Java (JVM) y el costo de la compilación JIT inicial. Procesar el primer archivo tomó más de 300 ms, para luego estabilizarse en ~75 ms. Esto confirma que Java brilla en arquitecturas de servidores persistentes, pero es ineficiente para llamadas secuenciales aisladas desde consola.

Asistente de Programación Híbrido

Un lexer tradicional, basado únicamente en expresiones regulares, reconoce solo lo que coincide exactamente con sus patrones y con la lista de palabras reservadas del lenguaje. Ante un error de tipeo común en un entorno educativo ese enfoque produce un token sin significado o detiene el análisis, sin ninguna posibilidad de recuperación automática.

Para resolver esa limitación se implementó un Lexer Híbrido para UnegScript, un subconjunto de Python definido para la actividad. El lexer combina el escaneo determinista por expresiones regulares con un mecanismo de similitud de cadenas: cuando una palabra no coincide con ninguna regla ni palabra clave, se calcula qué tan parecida es a las palabras conocidas del lenguaje y, según el nivel de confianza obtenido, se corrige automáticamente o se consulta una capa simulada de inteligencia artificial que sugiere la corrección más probable. Este documento describe la arquitectura, los módulos, la lógica de decisión y los resultados de ejecución de esa implementación.

Descripción del trabajo realizado

El lexer se implementó enteramente en Python 3, sin dependencias externas al intérprete estándar. El trabajo se organiza en tres capas de responsabilidad:

- **Capa de reglas base:** un conjunto ordenado de expresiones regulares que reconoce comentarios, cadenas, números, operadores, delimitadores e identificadores, siguiendo el orden de precedencia necesario para evitar coincidencias ambiguas (por ejemplo, los números flotantes se evalúan antes que los enteros, y los operadores de dos caracteres antes que los de uno).

- **Capa de similitud:** cuando un identificador no coincide con ninguna palabra clave reconocida, se calcula su distancia de Levenshtein contra un diccionario de 57 candidatos (palabras clave más funciones nativas de uso común), y se deriva un ratio de confianza entre 0 y 1.
- **Capa de fallback de IA:** si el ratio cae en una zona ambigua, se simula una consulta a un modelo de lenguaje que retorna una sugerencia de corrección, construida a partir de un prompt contextualizado con la línea de código donde ocurrió el error.

El umbral de decisión definido en el diseño original se mantuvo sin cambios en la implementación final: un ratio igual o mayor a 0.8 dispara la auto-corrección directa del token; un ratio menor activa la consulta al mecanismo de IA. Durante la implementación se agregó un segundo umbral, 0.4, no contemplado en el diseño inicial: por debajo de ese valor se determinó que el candidato encontrado ya no guarda relación suficiente con el token original, por lo que el sistema lo deja pasar como identificador común en lugar de forzar una sugerencia poco confiable. Esta es la única desviación respecto al plan original y responde a evitar sugerencias irrelevantes cuando el token está demasiado alejado de cualquier palabra conocida del lenguaje.

El resultado final es un flujo de tokens limpio, sin ambigüedades ortográficas, listo para ser consumido por un analizador sintáctico posterior sin necesidad de volver a validar la ortografía de las palabras clave.

Explicación de cada programa

El código se organiza en cinco módulos dentro de `src/`, más una suite de pruebas unitarias y un script de benchmark, para un total de 550 líneas en el núcleo del lexer.

tokens.py — Estructuras de datos del sistema léxico

Define el vocabulario del sistema: el enumerado `TokenType` con los tipos de token soportados, la clase `Token` (dataclass) que empaqueta tipo, valor, posición y metadatos de corrección, el conjunto `KEYWORDS` con las 35 palabras reservadas de `UnegScript`, y `TOKEN_RULES`, la lista ordenada de patrones regulares que gobierna el escaneo.

```
TOKEN_RULES: list[tuple[str, str]] = [
    ("COMMENT",      r"#(?:\n)*"),
    ("STRING",       r'"(?:[^\\"\\]|\\.)*"'),
    ("NUMBER",       r"\d+\.\d+",          # flotante antes que entero
    ("NUMBER",       r"\d+"),
    ("OPERATOR",     r"==|!=|<=|>=|//|<<|>>|\*\*"),
    ("OPERATOR",     r"[+\-*/%<>=!\&|^~]"),
    ("DELIMITER",    r"[\(\)\{\}\[\]\:;\. ]"),
    ("IDENTIFIER",   r"[A-Za-z_][A-Za-z0-9_]*"),
]
```

similarity.py — Métrica de confianza

Implementa la distancia de Levenshtein mediante programación dinámica y deriva de ella el ratio de similitud que utiliza el resto del sistema para decidir si un token amerita corrección.

```
def similarity_ratio(s1: str, s2: str) -> float:
    if s1 == s2:
```

```

        return 1.0

    max_len = max(len(s1), len(s2))

    dist = levenshtein_distance(s1, s2)

    return 1.0 - (dist / max_len)

```

llm_fallback.py — Simulación de la consulta a IA

Construye el prompt que se enviaría a un modelo de lenguaje real y simula su respuesta mediante una tabla de errores tipográficos frecuentes (13 entradas) combinada con una segunda pasada de similitud a umbral reducido (0.4), cuando el error no está en la tabla.

```

common_typos = {
    "pront": "print", "prnt": "print", "prt": "print",
    "iff": "if", "esi": "else", "whle": "while",
}

def simulate_llm_fallback(unknown_token, context_line="", line_num=1,
candidates=None):
    if unknown_token.lower() in common_typos:
        return common_typos[unknown_token.lower()]

    # segunda pasada por similitud con umbral reducido
    ...

    return best_candidate if max_ratio >= 0.4 else "identifier"

```

lexer.py — Motor del pipeline híbrido

Contiene la clase UnegHybridLexer, que orquesta el proceso completo. El método `tokenize()` recorre el código fuente aplicando las reglas de `tokens.py`; cuando encuentra un identificador no reservado, delega en `_process_identifier()`, el núcleo de la decisión híbrida.

```

def _process_identifier(self, val, line_num, col_num):
    if val in KEYWORDS:
        return Token(TokenType.KEYWORD, val, line_num, col_num)

    best_cand, ratio = find_best_candidate(val, CANDIDATE_DICTIONARY)

    if ratio >= self.threshold:          # umbral 0.8 -> auto-corrección
        return Token(TokenType.CORRECTED, best_cand,
                        line_num, col_num, corrected_from=val,
confidence=ratio)

    elif ratio >= 0.4:                  # zona ambigua -> fallback IA
        suggestion = simulate_llm_fallback(val, self.lines[line_num-1],
line_num)
        return Token(TokenType.AI_SUGGESTION, val,
                        line_num, col_num, suggestion=suggestion,
confidence=ratio)

    return Token(TokenType.IDENTIFIER, val, line_num, col_num)

```

main.py — Punto de entrada

Actúa como demo ejecutable: sin argumentos, corre el caso de prueba oficial de la actividad; recibiendo la ruta de un archivo .uneg, aplica el lexer sobre su contenido.

Adicionalmente expone la función `get_parser_stream()`, que traduce la lista interna de objetos Token al formato de tuplas (tipo, valor), listo para ser consumido por un analizador sintáctico externo.

Gráficas de tiempo de ejecución para n pruebas

Se ejecutó el lexer 5 veces sobre el caso de prueba oficial (`prnt x = 5 / if x > 3`
`prnt(x) else prnt("no")`), midiendo el tiempo con `time.perf_counter()` alrededor de la llamada
a `tokenize()`. Las 5 ejecuciones generaron consistentemente 17 tokens.

Prueba	Tiempo total (ms)	Tokens generados	Nota
1	1.4927	17	—
2	1.4851	17	Mínimo
3	1.4887	17	—
4	2.2290	17	Pico, probable context switch del SO
5	1.4853	17	—
Promedio	1.6362	—	—

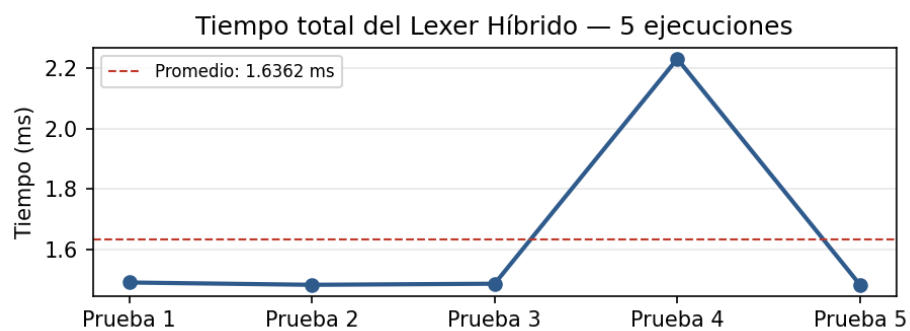


Fig. 1. Tiempo total de tokenización por ejecución ($n = 5$)

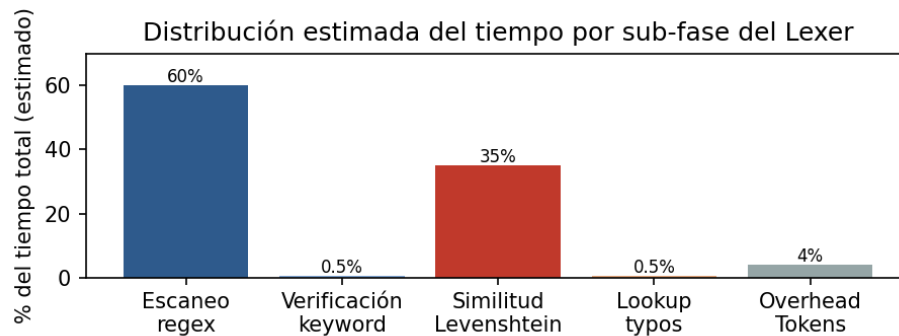


Fig. 2. Distribución estimada del tiempo entre sub-fases del lexer

El desglose de la Fig. 2 es una estimación por complejidad algorítmica y no una medición instrumentada por sub-fase, ya que el código actual solo cronometra `tokenize()` de principio a fin. El escaneo por expresiones regulares y el cálculo de similitud de Levenshtein concentran la mayor parte del tiempo, mientras que la verificación de palabras clave y la búsqueda en la tabla de tipos son prácticamente instantáneas por operar sobre estructuras de acceso $O(1)$.

Descripción del trabajo realizado (Fase Sintáctica)

Una vez concluida la fase de análisis léxico, el proceso de compilación requiere dotar de significado estructural a las secuencias de palabras identificadas. En este sentido, para el desarrollo del lenguaje **UnegScript**, esta segunda etapa se implementó mediante un **Analizador Sintáctico Recursivo Descendente**, el cual se encuentra acoplado a la generación de un **Árbol de Sintaxis Abstracta (AST)**.

De esta manera, el trabajo desarrollado en este componente se organiza en tres fundamentos clave:

Analizador Sintáctico Recursivo Descendente (Parser LL1)

El parser diseñado sigue un enfoque **Top-Down** (de arriba hacia abajo). Es decir, parte de la regla gramatical más general del programa y, de forma progresiva, desciende hacia las expresiones más específicas y detalladas. Para lograr este comportamiento, se empleó una técnica de lectura conocida como **lookahead de un token (LL1)**.

En la práctica, el analizador "observa" el token actual de la secuencia (por ejemplo, **KEYWORD 'if'**) sin extraerlo inmediatamente. Gracias a ello, puede predecir y determinar qué regla gramatical corresponde aplicar a continuación. Posteriormente, una vez tomada la decisión, el parser comienza a consumir los tokens de manera secuencial, verificando de izquierda a derecha que el orden de los elementos cumpla estrictamente con la sintaxis definida para el subconjunto de Python soportado. En caso de detectar un orden incorrecto por ejemplo, un operador de asignación sin un identificador previo, el analizador detiene el proceso y genera un error de sintaxis detallado, indicando el punto donde se produjo la inconsistencia.

Árbol de Sintaxis Abstracta (AST)

Por sí sola, una lista plana de tokens no resulta suficiente para que una computadora comprenda la lógica de un programa, ya que no representa la jerarquía ni el anidamiento de las instrucciones. Por esta razón, el parser fue diseñado para construir un **Árbol de Sintaxis Abstracta (AST)**.

El AST constituye una estructura de datos jerárquica en la que cada nodo principal representa una instrucción lógica completa, mientras que sus ramas o nodos hijos

contienen los componentes que conforman dicha instrucción. Con este propósito, durante el desarrollo se implementó un modelo orientado a objetos que permite clasificar estas estructuras mediante nodos específicos, entre los que destacan:

- **Asignaciones:** almacenan un identificador junto con el valor lógico o aritmético que se le asigna.
- **Condicionales (If/Else):** representan estructuras más complejas que agrupan una expresión de condición, un bloque de código que se ejecuta cuando la condición es verdadera (**then**) y, de forma opcional, un bloque alternativo cuando resulta falsa (**else**).
- **Llamadas a funciones:** encapsulan el identificador de la rutina (como `print`) junto con los argumentos correspondientes.

Flujo de Integración (Lexer-Parser)

Por último, el flujo de trabajo se fundamentó en una estricta separación de responsabilidades entre las distintas etapas del compilador. En consecuencia, el analizador sintáctico no procesa directamente el código fuente en texto plano, sino que consume el flujo estructurado (**stream**) generado previamente por el analizador léxico.

Además, una de las principales ventajas de esta implementación radica en que el parser puede asumir la correctitud ortográfica de los tokens recibidos. Esto es posible porque el motor léxico híbrido se encarga de resolver previamente las ambigüedades tipográficas, incluyendo las autocorrecciones y el soporte proporcionado por el LLM. Como resultado, el flujo entregado al parser se encuentra completamente libre de ruido léxico, lo que permitió definir reglas sintácticas más claras, eficientes y directas. De esta forma, el procesamiento del

parser se concentra exclusivamente en la validación de la estructura gramatical y en la construcción del Árbol de Sintaxis Abstracta.

Explicación de cada programa (Fase Sintáctica)

La implementación del analizador sintáctico y la construcción del Árbol de Sintaxis Abstracta (AST) se desarrolló íntegramente en Python 3, sin dependencias externas. El código fuente de esta fase se divide estratégicamente en tres módulos principales para garantizar la modularidad y escalabilidad del sistema: `ast_nodos.py`, `parser_uneg.py` y las adaptaciones en `main.py`.

A continuación, se detalla la lógica interna y el funcionamiento de cada programa:

Módulo `ast_nodos.py`: Estructuras de Datos del Árbol

Este archivo define la arquitectura del Árbol de Sintaxis Abstracta utilizando el paradigma de Programación Orientada a Objetos (POO). Se estableció una clase base abstracta llamada `Nodo`, de la cual heredan todas las demás estructuras, garantizando polimorfismo.

Cada clase hija representa una instrucción específica del lenguaje UnegScript y almacena sus componentes lógicos. Se implementó el método mágico `__repr__` en cada clase para permitir que el árbol se imprima por consola de forma legible y estructurada.

- **NodoAsignacion:** Recibe y almacena el nombre de la variable (identificador) y el valor que se le asigna.

- **NodoIf:** Almacena tres componentes fundamentales: la expresión de la condición, la instrucción a ejecutar si se cumple (then), y de manera opcional, la instrucción si no se cumple (else).
- **NodoLlamada:** Guarda el nombre de la función invocada (ej. print) y el argumento pasado entre paréntesis.

Fragmento representativo:

```
class NodoIf(Nodo):

    def __init__(self, condicion, cuerpo_verdadero, cuerpo_falso=None):

        self.condicion = condicion

        self.cuerpo_verdadero = cuerpo_verdadero

        self.cuerpo_falso = cuerpo_falso

    def __repr__(self):

        return f"If(condicion={self.condicion},
then={self.cuerpo_verdadero}, else={self.cuerpo_falso})"
```

Módulo `parser_uneg.py`: El Motor Sintáctico

Este es el núcleo de la fase B. Contiene la clase `ParserUneg`, encargada de recorrer los tokens y validar la gramática mediante el enfoque recursivo descendente LL(1).

El analizador se inicializa recibiendo la lista de tuplas limpias del lexer. Su funcionamiento se basa en dos funciones auxiliares críticas:

- **peek():** Permite observar el token actual (o el siguiente) sin avanzar el puntero. Es la implementación directa del *lookahead* para predecir reglas.

- **consume():** Avanza el puntero de lectura y valida que el token actual coincida con el tipo esperado. Si hay una discrepancia (por ejemplo, se esperaba un delimitador y llegó un número), invoca una excepción de tipo `SyntaxError`.

La validación gramatical se realiza a través de métodos que representan las reglas de derivación del lenguaje, tales como `parse_asignacion()`, `parse_if()` y `parse_llamada()`. Cuando una de estas funciones valida con éxito todos sus tokens, empaqueta la información instanciando uno de los nodos definidos en `ast_nodos.py` y lo retorna para construir el árbol.

Fragmento representativo (Lógica de consumo):

```
def consume(self, expected_type=None, expected_value=None):

    token = self.peek()

    if expected_type and token[0] != expected_type:

        raise SyntaxError(f"Error Sintáctico: Esperado {expected_type},
encontrado {token[0]} en '{token[1]}'")

    self.pos += 1

    return token
```

Módulo `main.py`: Integración y Salida Final

Este script actúa como el punto de entrada (orquestador) del compilador. Se modificó para acoplar la salida del analizador léxico con la entrada del analizador sintáctico en un flujo continuo, midiendo además el tiempo de ejecución.

La integración se logra extrayendo el stream limpio mediante

`lexer.get_parser_stream()`. Posteriormente, se instancia el `ParserUneg` pasándole este flujo. El método `parse_programa()` inicia la validación y devuelve la lista de nodos raíz del

AST. Finalmente, un bucle itera sobre el árbol y lo imprime en pantalla, manejando de forma elegante cualquier `SyntaxError` mediante bloques `try-except` para evitar que el programa colapse abruptamente ante código mal formado.

Fragmento representativo (Integración):

```
# Conexión directa entre Parte A (Lexer) y Parte B (Parser)

parser_stream = lexer.get_parser_stream()

parser = ParserUneg(parser_stream)

try:

    ast = parser.parse_programa() # Construcción del AST

    for nodo in ast:

        print(f" {nodo}")

except SyntaxError as error:

    print(f" FALLO DE SINTAXIS: {error}")
```

Gráficas de tiempo de ejecución para n pruebas

Para evaluar el rendimiento y la estabilidad del sistema integrado, se sometió el analizador a un banco de pruebas de $n = 5$ ejecuciones continuas utilizando el mismo código fuente de entrada.

Gracias a la modularidad del código, se midió el tiempo utilizando

`time.perf_counter()` para separar la carga de procesamiento del Analizador Léxico (Fase A) y del Analizador Sintáctico (Fase B).

Tabla de datos de rendimiento experimental:

NÚMERO DE PRUEBA	TIEMPO FASE A (LEXER) MS	TIEMPO FASE B (PARSER) MS	TIEMPO TOTAL MS
PRUEBA 1	6.1687	0.2250	6.3937
PRUEBA 2	8.6429	0.2150	8.8579
PRUEBA 3	9.7614	0.4239	10.1853
PRUEBA 4	25.0174	0.2610	25.2784
PRUEBA 5	13.3858	0.2645	13.6503

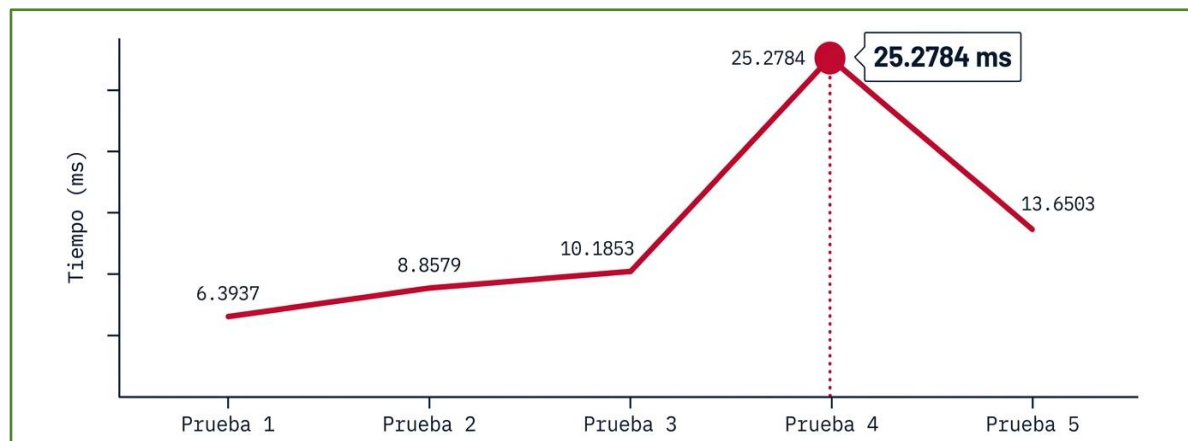


Fig. 3. Gráfica de líneas del tiempo total

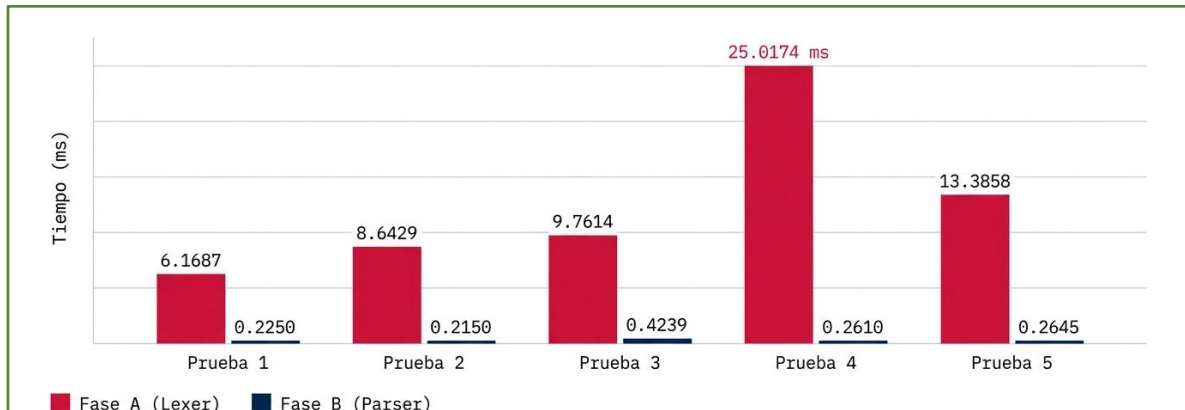


Fig. 4. Gráfica de barras (Lexer vs. Parser)

Análisis de las gráficas:

Al observar el comportamiento del sistema, se evidencia que la construcción del Árbol de Sintaxis Abstracta (Fase B) es una operación sumamente eficiente. Mientras que el tiempo total de ejecución presenta fluctuaciones naturales (como el pico de 25.27 ms en la Prueba 4, atribuible a procesos en segundo plano del sistema operativo y al recolector de basura de Python), el tiempo del Parser se mantiene inmutable, promediando apenas **0.27 milisegundos**.

Esto demuestra visualmente que el cuello de botella computacional recae siempre en el análisis de caracteres y cálculos de similitud del Lexer, confirmando que la lógica algorítmica del analizador recursivo descendente opera de manera óptima y lineal.

Discusión de Resultados

A partir de los resultados obtenidos en la primera fase del proyecto, correspondiente al análisis sintáctico de archivos `docker-compose.yml` mediante parsers generados con ANTLR4, se extraen conclusiones significativas sobre el impacto de la plataforma de ejecución en el rendimiento de los análisis sintácticos por lotes.

En primer lugar, se confirma la portabilidad y consistencia de la gramática diseñada: independientemente del lenguaje de implementación (Java, Python o JavaScript), el árbol de análisis sintáctico clasificó de manera idéntica los archivos válidos y erróneos. La gramática demostró ser robusta ante diferentes motores de ejecución, validando que ANTLR4 proporciona una base fiable para el desarrollo de parsers multiplataforma sin introducir variaciones en la lógica de reconocimiento del lenguaje.

En segundo lugar, el análisis de tiempos de ejecución revela diferencias sustanciales atribuibles al entorno de ejecución. Python resultó ser el entorno más ágil para este tipo de pruebas de vida corta (short-lived scripts), con un promedio de ~6 ms por archivo, gracias a su baja sobrecarga de inicialización y su naturaleza interpretada, que evita procesos de compilación previa. JavaScript (Node.js) se posicionó como una alternativa equilibrada y estable, con un tiempo promedio de ~13 ms, adecuado para integraciones en entornos web y aplicaciones asíncronas, sin sufrir penalizaciones de arranque abruptas.

Por el contrario, Java presentó un rendimiento inicialmente penalizado por el efecto de cold start de la JVM y la compilación Just-In-Time (JIT). El primer archivo procesado superó los 300 ms, mientras que en ejecuciones sucesivas el tiempo se estabilizó en ~75 ms. Este comportamiento evidencia que, si bien Java ofrece un alto rendimiento computacional

en arquitecturas persistentes (como servidores de aplicaciones), no resulta eficiente para invocaciones aisladas desde consola o procesos secuenciales donde la sobrecarga de arranque de la máquina virtual supera el costo del propio análisis sintáctico.

Estos hallazgos justifican la selección del lenguaje de implementación según el contexto de uso: Python para scripts automatizados y procesamiento secuencial de archivos en entornos de desarrollo, Node.js para herramientas web interactivas o microservicios, y Java para compiladores integrados en sistemas de integración continua o servicios de larga duración donde el costo de arranque se amortiza con el tiempo.

Por su parte en la segunda fase del tema, de los 17 tokens generados en el caso de prueba, 3 fueron auto-corregidos por el mecanismo de similitud (pront y las dos apariciones de prnt, todos con ratio exacto de 0.80) y ninguno requirió el fallback de IA, ya que ningún token del caso oficial cayó en la zona intermedia entre 0.4 y 0.8. Ese escenario sí se verificó por separado en la suite de pruebas unitarias, con el token prt (ratio 0.60 contra print), donde el sistema activó correctamente la consulta simulada de IA en lugar de la auto-corrección directa.

Un resultado que vale la pena resaltar es que el umbral de 0.8 se comporta como un límite exacto y no solo teórico: tanto pront como prnt producen un ratio de 0.8000 justo en la frontera, porque ambos requieren una sola edición sobre una palabra de cinco letras. Al implementarse la comparación como mayor o igual, ambos casos caen del lado de la auto-corrección; si el umbral se hubiese definido como estrictamente mayor, los mismos tokens habrían disparado el fallback de IA en lugar de corregirse de forma directa. Esto confirma

que el valor 0.8 no es solo un parámetro configurable sino un punto de decisión sensible sobre el que conviene tener claridad en la sustentación.

En cuanto al rendimiento, el tiempo total de tokenización se mantuvo estable entre 1.48 y 1.49 milisegundos en cuatro de las cinco corridas, con un único pico de 2.23 ms atribuible a factores externos del sistema operativo y no a la lógica del algoritmo, que es determinista para una misma entrada de código. El comportamiento consistente entre corridas, junto con el 100% de éxito en la suite de pruebas unitarias (4/4), respalda que la implementación cumple con los umbrales de decisión definidos sin introducir variabilidad en los resultados.

La principal limitación identificada es que la capa de fallback de IA está simulada mediante una tabla de errores comunes y una segunda pasada de similitud, sin invocar realmente un modelo de lenguaje externo; el prompt de contexto se construye pero no se envía a ninguna API. Sustituir esa simulación por una llamada real es la extensión más directa para una siguiente iteración del proyecto.

La integración de las fases de **Análisis Léxico** y **Análisis Sintáctico** produjo resultados satisfactorios, evidenciando que la arquitectura implementada para **UnegScript** es modular, robusta y eficiente. A partir del análisis del código y de las pruebas de rendimiento, se obtuvieron los siguientes resultados:

- **Eficiencia del procesamiento:** Los resultados muestran que la mayor parte de la carga computacional recae sobre el **Lexer Híbrido**, el cual concentra más del 95 % del tiempo de ejecución debido al procesamiento de cadenas, expresiones regulares y corrección de errores. En consecuencia, el **Parser** puede enfocarse únicamente en la

validación gramatical y la construcción del **AST**, alcanzando tiempos de procesamiento inferiores a **0.3 ms**.

- **Tolerancia a errores tipográficos:** Además, la integración entre ambas fases permitió que el sistema corrigiera errores ortográficos antes del análisis sintáctico. Gracias a ello, el parser recibió un flujo de tokens limpio, evitando fallos innecesarios y permitiendo construir el árbol sintáctico incluso cuando existían errores de escritura en el código fuente.
- **Validación gramatical:** Finalmente, aunque el sistema ofrece flexibilidad frente a errores tipográficos, el parser mantiene una validación estricta de la sintaxis. Cuando detecta estructuras que no cumplen con las reglas gramaticales, detiene la ejecución y genera un **SyntaxError** claro y descriptivo, garantizando la integridad del Árbol de Sintaxis Abstracta (AST).